DIGIZAFE — MEMBER 4 MASTER TECHNICAL / VIVA NOTES

Your role

Zero-Trust Credential Protection & Secure Password Vault Engineer

Your contribution can be framed as:

> **"I worked on the secure credential-protection side of DigiZafe, focusing on the Zero-Trust password vault, client-side encryption, secure credential handling, password exposure verification, and privacy-preserving password checking."**

The research paper explicitly identifies client-side PBKDF2-derived AES-GCM encryption and k-anonymity-based password exposure verification as core DigiZafe security mechanisms.

PART I — WHAT PROBLEM YOU SOLVE

DigiZafe isn't only supposed to tell the user:

> **"Your password may be exposed."**

It also needs to answer:

> **"What should you do about it securely?"**

The project's research gap is specifically the fragmentation between:

Exposure Detection
↓
Risk Interpretation
↓
Password Mitigation

DigiZafe brings those together in one workflow.

Your part is primarily the final security layer:

Exposure
↓
Risk
↓
Generate stronger credential
↓
Encrypt locally
↓
Store securely
↓
Retrieve only when authorized

PART II — ZERO-TRUST VAULT

The central principle is:

> **The backend should not need access to plaintext vault credentials.**

The architecture therefore uses:

User Password / Master Secret
↓
PBKDF2
↓
Derived Encryption Key
↓
AES-GCM
↓
Encrypted Vault Data
↓
Backend

The paper explicitly states that sensitive vault contents are encrypted on the client using PBKDF2-derived AES-GCM keys before transmission, preventing backend services from accessing plaintext credentials.

PART III — WHY CLIENT-SIDE ENCRYPTION?

Imagine the conventional architecture:

Browser
↓
plaintext password
↓
Backend
↓
Database

If the backend is compromised, plaintext credentials may become exposed.

DigiZafe instead aims for:

Browser
↓
plaintext credential
↓
LOCAL ENCRYPTION
↓
ciphertext
↓
Backend
↓
Database

So the server handles ciphertext rather than the plaintext vault contents.

PART IV — WHAT IS PBKDF2?

PBKDF2 means:

Password-Based Key Derivation Function 2

Its job is to transform a password/passphrase into a cryptographic key.

Conceptually:

Master Password
↓
PBKDF2
↓
Derived Key

It deliberately makes key derivation computationally expensive.

This makes brute-force password guessing more expensive than using the raw password directly as an encryption key.

PART V — WHAT IS A SALT?

PBKDF2 should use a random salt.

Conceptually:

Password
+
Random Salt
↓
PBKDF2
↓
Encryption Key

Why?

Without a unique salt, two users using the same password could derive identical keys, and precomputed attacks become easier.

PART VI — WHY AES-GCM?

DigiZafe uses:

AES-GCM

AES provides:

Confidentiality

GCM additionally provides:

Integrity/authentication

So encryption doesn't just hide the vault data; it also allows tampering to be detected.

Conceptually:

Plaintext
+
Key
+
Nonce
↓
AES-GCM
↓
Ciphertext + Authentication Tag

The research material specifically identifies AES-GCM as the authenticated encryption mechanism used for the vault.

PART VII — AES-GCM COMPONENTS

You should know these terms:

Key

Secret cryptographic material.

Nonce / IV

A unique value used for an encryption operation.

Ciphertext

Encrypted plaintext.

Authentication tag

Used to verify that ciphertext was not modified.

So conceptually:

Plaintext
↓

AES-GCM(Key, Nonce)
  ↓
Ciphertext
+
Authentication Tag

PART VIII — WHY NONCE UNIQUENESS MATTERS

Professor:

> "Can you reuse the same nonce with the same AES-GCM key?"

Answer:

> **No. AES-GCM requires nonce uniqueness for a given key. Reusing a nonce can seriously compromise security.**

This is one of the most important cryptographic implementation details to understand.

PART IX — WHAT DOES ZERO-TRUST MEAN HERE?

Don't say:

> "Zero trust means nobody is trusted."

That's too vague.

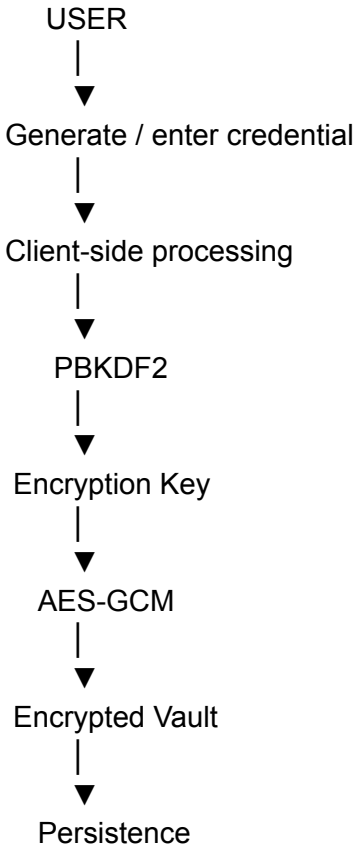
For DigiZafe:

> **The credential-protection architecture minimizes trust in backend services by ensuring that sensitive vault plaintext remains client-side and only encrypted data needs to be persisted or transmitted.**

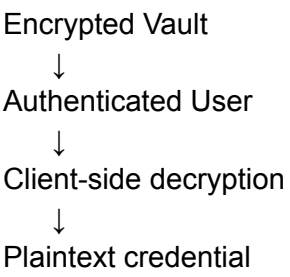
The paper explicitly describes this as a Zero-Trust credential-management architecture.

PART X — VAULT WORKFLOW

The complete workflow:



Retrieval:



The research implementation describes the vault as responsible for encryption, storage, retrieval and deletion of credentials.

PART XI — WHY NOT STORE PLAINTEXT PASSWORDS?

Because a database compromise would immediately expose credentials.

Bad architecture:

Database
————————————
username
password
————————————

Better:

Database
————————————————————
encrypted_vault_blob
nonce
authentication data
————————————————————

The backend doesn't need the plaintext vault contents.

PART XII — PASSWORD GENERATION

DigiZafe also includes password generation.

The research paper describes the password suite as generating high-entropy passwords and storing credentials through authenticated encryption.

A good conceptual generation pipeline is:

Secure Randomness
↓
Random characters
↓
High-entropy password
↓
User accepts
↓
Encrypt

↓
Vault

The important word is:

Cryptographically secure randomness

—not merely:

random.randint(...)

for security-sensitive password generation.

PART XIII — WHY PASSWORD GENERATION IS PART OF DIGIZAFE

The platform isn't just:

Detect breach

It follows:

Detect
↓
Understand
↓
Mitigate

The research paper explicitly identifies the combination of exposure detection, explainable risk interpretation and password mitigation as the project's research gap.

PART XIV — VAULT HEALTH

The system can help users identify:

Weak password
Reused password

Exposed password

and then recommend:

Generate stronger password

The paper specifically describes vault-health analysis for weak or reused credentials.

PART XV — PASSWORD REUSE

Suppose:

Gmail → Password A
GitHub → Password A
Instagram → Password A

One compromise can increase the impact across multiple services.

So:

Password reuse
↓
Higher credential vulnerability

The project's risk model explicitly incorporates repeated credential compromise/reuse into credential vulnerability.

PART XVI — PASSWORD EXPOSURE VERIFICATION

This is another major part of your area.

DigiZafe doesn't simply send the complete password to an external breach service.

Instead it uses a:

k-anonymity privacy model

The research paper explicitly identifies this as a privacy-preserving password-verification mechanism.

PART XVII — WHAT IS K-ANONYMITY IN PASSWORD CHECKING?

The basic idea:

Password
↓
Hash
↓
Only partial hash information
↓
External service

The external service doesn't need the complete password hash.

The paper specifically describes the Pwned Passwords approach as sending only a small SHA-1 hash prefix rather than the complete hash.

PART XVIII — WHY IS THIS IMPORTANT?

Bad approach:

Password
↓
Full hash
↓
External API

Better privacy approach:

Password
↓
SHA-1 hash

↓
Prefix
↓
External API

The external service sees only partial information.

This reduces unnecessary information disclosure.

PART XIX — IMPORTANT:

K-ANONYMITY ≠ ENCRYPTION

Professor:

> "Is k-anonymity encryption?"

Answer:

> **No. It is a privacy-preserving query technique. It reduces what information is disclosed to the external service; it does not encrypt the password itself.**

This distinction is essential.

PART XX — COMPLETE PASSWORD EXPOSURE FLOW

User Password
↓
Hash
↓
Prefix extraction
↓
External breach service
↓
Candidate hash suffixes
↓

Local comparison
↓
Exposed / Not exposed

The full password does not need to be sent.

PART XXI — WHY SHA-1 IF IT IS CRYPTOGRAPHICALLY OLD?

This is a tricky viva question.

Answer:

> **"SHA-1 is not being used here as a modern password-storage primitive. In the k-anonymity breach-checking protocol, it is used as the identifier format for querying a precomputed password corpus while only exposing a prefix. The security objective is minimizing disclosure during the query, not using SHA-1 as a password encryption mechanism."**

That's a much better answer than saying SHA-1 is "secure."

PART XXII — VAULT AUTHENTICATION VS VAULT ENCRYPTION

Do not confuse these.

Authentication

Answers:

> **Who are you?**

The platform's account authentication uses password verification and can use MFA. The functional requirements specify Argon2id verification and optional TOTP/recovery-code MFA.

Encryption

Answers:

Can someone read the stored vault data?

The vault uses client-side PBKDF2-derived AES-GCM encryption.

So:

Authentication
↓
Identity/session

Encryption
↓
Data confidentiality

PART XXIII — ARGON2ID VS PBKDF2

This is an excellent viva question.

Argon2id

Used for:

Account password authentication/storage.

PBKDF2

Used in the documented vault architecture for:

Deriving the encryption key for the encrypted vault.

The functional requirements specify Argon2id for credential verification, while the research architecture describes PBKDF2-derived AES-GCM for vault encryption.

Don't say:

"We use PBKDF2 to store login passwords."

That would conflate two different mechanisms.

PART XXIV — MFA

The authentication layer also supports:

Password
+
TOTP

or recovery codes.

The functional requirements specify:

- TOTP-based MFA
- recovery codes
- step-up reauthentication for sensitive MFA changes
- single-use recovery codes
- session revocation on security-sensitive actions.

PART XXV — WHY MFA?

Even if the password is compromised:

Attacker has password
↓
Needs second factor
↓
TOTP / recovery mechanism

This reduces account takeover risk.

PART XXVI — RECOVERY CODES

Recovery codes should be:

generated
hashed for storage
single-use
not displayed again

The requirements explicitly state that recovery codes are hashed and never re-displayed.

PART XXVII — SESSION SECURITY

The authentication system uses:

Access token
+
Rotating refresh token

with the refresh token stored in an HttpOnly cookie according to the functional requirements.

This helps reduce exposure of refresh credentials to ordinary client-side script access.

PART XXVIII — REFRESH TOKEN ROTATION

Conceptually:

Login
↓
Access Token A
Refresh Token A
↓
Refresh
↓
Access Token B
Refresh Token B
↓
Token A invalidated

If an old rotated refresh token is reused:

TOKEN_REUSE_DETECTED

and the refresh-token family is revoked according to the requirements.

PART XXIX — ACCOUNT SECURITY FLOW

Register
↓
Email Verification
↓
Login
↓
Password Verification
↓
MFA if enabled
↓
Session
↓
Access Token + Refresh Token

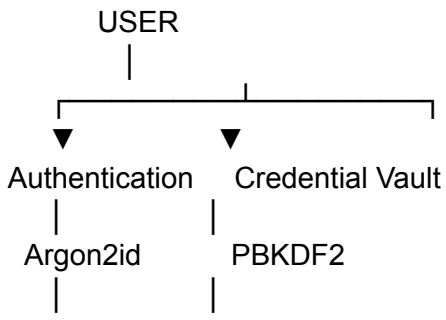
Logout:

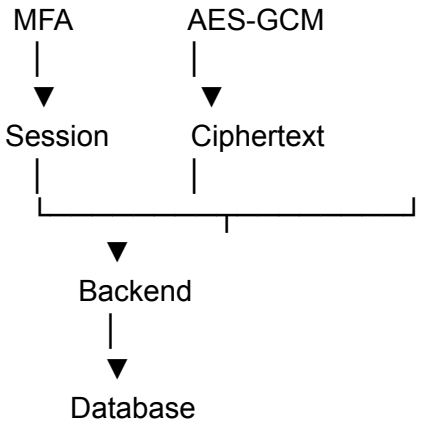
Logout
↓
Refresh family revoked
↓
Cookie cleared

The functional requirements explicitly define these lifecycle behaviors.

PART XXX — YOUR SECURITY BOUNDARY

Your area can be visualized as:





The important Zero-Trust principle is:

Backend
↓
does NOT need
↓
vault plaintext

PART XXXI — WHERE YOUR COMPONENT CONNECTS TO RISK ENGINE

Your module doesn't exist independently.

The overall pipeline is:

Exposure Detection
↓
Risk Assessment
↓
Credential Vulnerability
↓
Recommendation
↓
Password Generation
↓
Secure Vault

The research model explicitly separates:

Identity Visibility

from:

Credential Vulnerability

before producing the Digital Safety Index.

PART XXXII — CREDENTIAL VULNERABILITY

The model considers:

Password exposure
Breach frequency
Temporal recency
Repeated compromise
Breach severity

So your security module supplies the mitigation side of this problem.

PART XXXIII — DUAL-AXIS MODEL

The research architecture defines:

Axis 1

Identity Visibility

Measures:

> How publicly observable the identity is.

Axis 2

Credential Vulnerability

Measures:

> How vulnerable the authentication credentials are.

Then they are combined into:

Digital Safety Index

The paper explicitly defines the dual-axis methodology.

PART XXXIV — DSI FORMULA

The documented research model gives:

DSI = 100 − (0.40 × Videntity + 0.60 × Vcred)

where:

Videntity = identity visibility
Vcred = credential vulnerability

Credential vulnerability receives the larger weight because exposed authentication material is treated as the more immediate threat.

PART XXXV — WHY 60% CREDENTIAL VULNERABILITY?

Professor:

> "Why give credential vulnerability a higher weight?"

Answer:

> **"Because the research model treats compromised authentication material as generally presenting a more immediate security threat than passive public visibility, so credential vulnerability receives greater weight in the composite DSI."**

PART XXXVI — TEMPORAL DECAY

The credential vulnerability model includes temporal recency.

The documented equation is:

$T = \max(F, 1 - \lambda A)$

where:

A = age of newest breach
λ = decay coefficient
F = minimum decay floor

This means older exposure can have reduced influence while not necessarily disappearing completely.

PART XXXVII — CREDENTIAL VULNERABILITY FORMULA

The documented model gives:

$V_{cred} = (B + M) \times T \times R$

where:

B = exposure severity weight
M = breach surface/frequency multiplier
T = temporal decay
R = repeated compromise/reuse multiplier

PART XXXVIII — WHY THIS IS BETTER THAN A BREACH COUNT

Bad:

User A → 1 breach
User B → 10 breaches

and simply saying:

B has 10× the risk

is too simplistic.

DigiZafe considers:

severity
+
frequency
+
recency
+
repeated compromise

The research explicitly positions the model as more than a simple breach counter.

PART XXXIX — EXPLAINABILITY

A user shouldn't just see:

DSI = 37

They should understand:

Why?

The system therefore produces deterministic explanations and actionable security guidance.

Example:

High credential vulnerability

Reasons:
• recent breach exposure
• repeated credential compromise
• high breach severity

Recommended:
Generate a new password
Avoid reuse
Store it in the encrypted vault

PART XL — WHY DETERMINISTIC SCORING?

The presentation explicitly emphasizes deterministic scoring because it guarantees explainability.

Good viva answer:

> **"We chose deterministic scoring for the Digital Safety Index because users and evaluators need to understand exactly which exposure factors caused the resulting risk level. A deterministic model also makes the assessment reproducible and auditable."**

PART XLI — HOW YOUR MODULE CONNECTS TO THE DASHBOARD

The user sees:

Investigation
↓
Risk Score
↓
Credential vulnerability
↓
Recommendation
↓
Password generator
↓
Vault

The research paper describes the final dashboard as presenting exposure indicators, recommendations and credential-management capabilities.

PART XLII — YOUR END-TO-END FLOW

Memorize this:

User investigates identifier
↓
Threat intelligence collection
↓
Telemetry normalization
↓
Identity visibility
+
Credential vulnerability
↓
Digital Safety Index
↓
Explainable recommendation
↓
"Password should be changed"
↓
Secure password generation
↓
Client-side encryption
↓
PBKDF2-derived key
↓
AES-GCM encryption
↓
Encrypted vault

This connects your work directly to the complete DigiZafe architecture.

PART XLIII — 30 QUESTIONS YOU MUST MASTER

Q1. What is your contribution?

"My main contribution is the credential-security and Zero-Trust protection layer, including secure password generation, client-side encrypted vault protection, privacy-preserving password exposure verification, and the security controls surrounding authentication and credential handling."

Q2. Why client-side encryption?

To ensure backend services do not need access to plaintext vault contents.

Q3. What encryption algorithm?

AES-GCM.

Q4. Why AES-GCM?

It provides authenticated encryption: confidentiality plus integrity/authentication.

Q5. What is PBKDF2?

A password-based key derivation function used to derive an encryption key from a password/passphrase.

Q6. Why not directly use the password as the AES key?

Because passwords have variable and potentially weak entropy. A KDF such as PBKDF2 derives a cryptographic key with controlled computational cost.

Q7. What is a salt?

A random value combined with the password during key derivation to make derived keys unique and resist precomputed attacks.

Q8. What is a nonce?

A unique value used with AES-GCM for an encryption operation.

Q9. Can an AES-GCM nonce be reused?

Not with the same key. Nonce reuse can compromise security.

Q10. What is ciphertext?

The encrypted representation of plaintext.

Q11. What is the authentication tag?

Data used by AES-GCM to detect tampering and authenticate the ciphertext.

Q12. What is k-anonymity password checking?

A privacy-preserving breach-checking technique where only partial hash information is sent to the external service rather than the complete password or complete hash.

Q13. Is SHA-1 used to securely store passwords?

No. In the documented breach-checking workflow, it is used as part of the range-query mechanism; it is not the password-storage mechanism.

Q14. What is Argon2id used for?

Account password verification/authentication.

The functional requirements explicitly specify Argon2id verification.

Q15. PBKDF2 vs Argon2id?

Argon2id is used for account-password authentication in the specified requirements, while PBKDF2 is used for deriving the vault encryption key.

Q16. Why MFA?

To add another authentication factor so possession of the account password alone is insufficient.

Q17. What happens if an old refresh token is reused?

The requirements specify detecting token reuse, revoking the refresh-token family and auditing the event.

Q18. What is password reuse?

Using the same credential across multiple services, increasing the impact of one compromise.

Q19. Why generate passwords?

To provide a practical mitigation after exposure is detected.

Q20. Why is the vault important?

Detection without remediation leaves the user exposed. The vault provides a secure destination for replacement credentials.

Q21. What is Zero Trust in your implementation?

Minimize trust in backend services by keeping sensitive vault plaintext protected on the client.

Q22. Why not store plaintext vault data?

A database or backend compromise could directly expose all stored credentials.

Q23. Why deterministic risk scoring?

Reproducibility, explainability and auditability.

Q24. What are the two risk axes?

Identity Visibility and Credential Vulnerability.

Q25. What factors affect credential vulnerability?

Password exposure, breach frequency, temporal recency, repeated compromise/reuse and breach severity.

Q26. What is DSI?

Digital Safety Index — the composite score produced from identity visibility and credential vulnerability.

Q27. What is the DSI formula?

DSI = 100 − (0.40Videntity + 0.60Vcred)

Q28. Why does credential vulnerability have a larger weight?

> Because the research model treats exposed authentication material as generally more immediately threatening than passive identity visibility.

Q29. What happens after a high-risk result?

> DigiZafe provides explainable, actionable recommendations, including credential mitigation such as generating a stronger password and using the secure vault.

Q30. What is the biggest limitation?

> The documented prototype/research architecture is designed around controlled, privacy-preserving exposure verification and does not claim that any risk score represents absolute real-world security. The research prototype also emphasizes reproducibility and avoids handling real compromised credential pairs.

PART XLIV — 15 HARD PROFESSOR QUESTIONS

1. Why PBKDF2 instead of simply hashing the master password?

Because a hash and a KDF serve different purposes. PBKDF2 deliberately derives a key with configurable computational work and uses a salt.

---

2. Why AES-GCM instead of AES-CBC?

AES-GCM provides authenticated encryption, whereas CBC by itself provides confidentiality but requires a separate integrity mechanism.

---

3. What happens if ciphertext is modified?

AES-GCM authentication should fail rather than silently returning corrupted plaintext.

---

4. What happens if the user's master password is forgotten?

The encryption key cannot simply be recovered from the backend if the architecture correctly keeps the vault encrypted and the key client-derived. This is the fundamental trade-off of strong client-side encryption.

---

5. Does the backend ever see the plaintext password?

For the vault architecture, the design goal is **no**; sensitive vault contents are encrypted client-side before transmission.

---

6. Does AES-GCM hide the password from the user?

No. It protects stored/transmitted vault data. The authenticated user can decrypt it client-side when authorized.

---

7. Why is nonce reuse dangerous?

Because AES-GCM's security assumptions depend on nonce uniqueness for a given key.

---

8. Is k-anonymity the same as encryption?

No. It reduces query disclosure; it is not an encryption algorithm.

---

9. Why doesn't DigiZafe send the complete password to a breach API?

Because doing so unnecessarily discloses highly sensitive information. The documented range-query approach uses partial hash information.

---

10. Why not use only breach count for risk?

Because two breaches can have very different severity, recency and reuse implications.

---

11. Why include temporal decay?

Because recent exposure generally represents a different risk state from very old exposure, while the model can retain a minimum influence through its decay floor.

---

12. Why include repeated compromise?

Because repeated exposure or reuse increases the potential impact of credential compromise.

---

13. Why not use ML for DSI?

The research presentation emphasizes deterministic scoring because explainability and reproducibility are central to the safety index.

---

14. What happens after the system identifies a vulnerable password?

The user can be guided toward mitigation, including generating a stronger credential and storing it in the encrypted vault.

---

15. What makes DigiZafe different from a password manager?

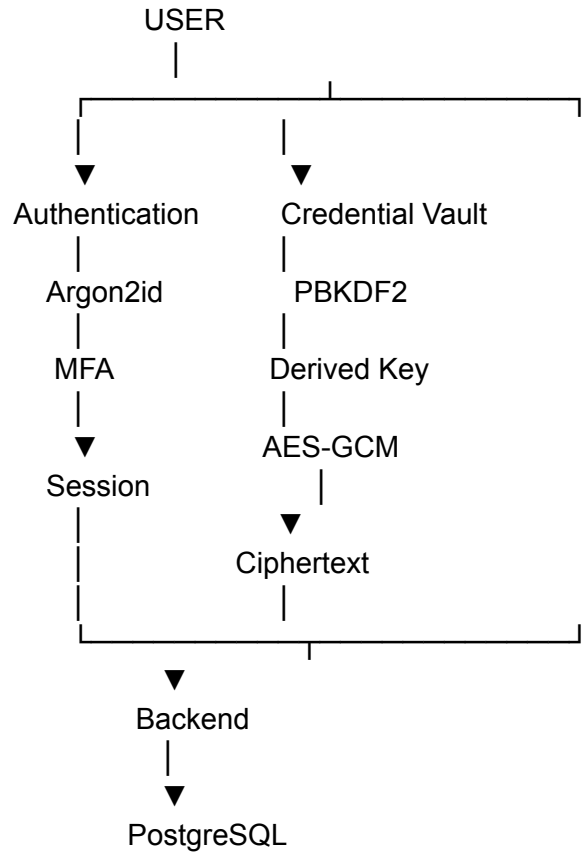
A password manager primarily protects credentials.

DigiZafe connects:

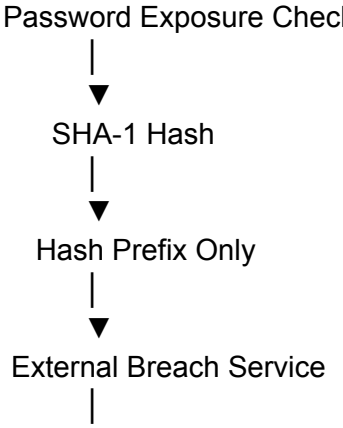
Exposure
+
Risk assessment
+
OSINT
+
Credential protection
+
Remediation

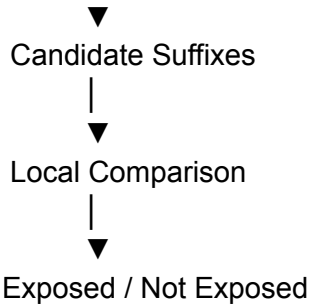
The research comparison explicitly positions DigiZafe as integrating breach detection, OSINT aggregation, explainable risk assessment, password vault protection and integrated remediation.

PART XLV — THE ARCHITECTURE YOU SHOULD DRAW



And alongside it:





PART XLVI — YOUR 5-MINUTE VIVA ANSWER

If the professor asks:

> **"Explain your contribution to DigiZafe."**

Say:

> **"My main contribution was on the credential-protection and privacy side of DigiZafe. The key problem we addressed was that detecting credential exposure is not enough; the user also needs a secure way to replace and protect credentials without requiring the backend to see plaintext vault data.**
>
> **We therefore designed a Zero-Trust credential-management workflow where sensitive vault contents are encrypted client-side using AES-GCM, with the encryption key derived using PBKDF2. This means the backend can persist encrypted vault data without requiring access to the plaintext credentials.**
>
> **The system also includes password generation and vault-health analysis so that exposed, weak or reused credentials can be replaced with stronger ones. For password exposure verification, we use a privacy-preserving k-anonymity approach where the complete password is not transmitted to an external breach service; instead, a hash prefix is used for the range query and the returned candidates are compared locally.**
>
> **At the risk-assessment level, DigiZafe separates identity visibility from credential vulnerability. Credential vulnerability considers factors such as exposure severity, breach frequency, recency and repeated compromise. These are combined into the deterministic Digital Safety Index, which provides an interpretable security signal and actionable remediation guidance.**

The main security principle behind my contribution is that exposure analysis should lead to remediation without creating another credential-storage risk. So the system connects risk detection to password generation and secure, client-side encrypted credential protection."

PART XLVII — 60-SECOND VERSION

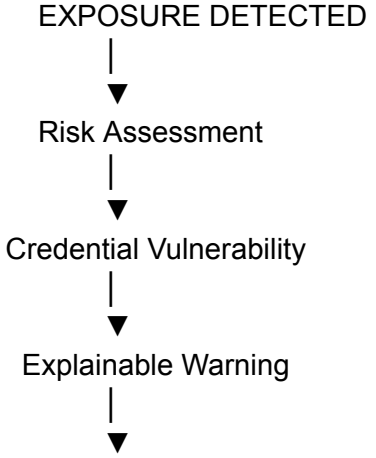
Memorize this one:

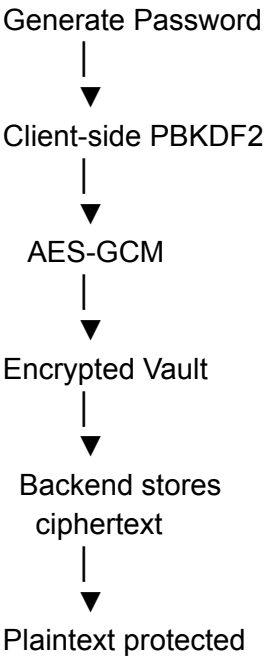
"I worked primarily on DigiZafe's Zero-Trust credential-protection layer. The goal was to connect exposure detection with secure password remediation without exposing plaintext vault credentials to backend services. We use PBKDF2 for key derivation and AES-GCM for authenticated client-side vault encryption.

The system also supports high-entropy password generation and vault-health analysis for weak or reused credentials. For password breach verification, we use a k-anonymity range-query approach so the complete password isn't disclosed to the external service.

This integrates with DigiZafe's deterministic risk engine, where credential vulnerability considers breach severity, frequency, recency and repeated compromise. The result is an explainable Digital Safety Index and actionable remediation workflow."

PART XLVIII — YOUR MOST IMPORTANT DIAGRAM





PART XLIX — 10 RULES TO REMEMBER

🔴 Rule 1

Never store vault plaintext on the backend.

🔴 Rule 2

PBKDF2 derives the encryption key; AES-GCM encrypts the vault.

🔴 Rule 3

Argon2id is for account-password authentication in the specified requirements.

🔴 Rule 4

AES-GCM provides authenticated encryption.

🔴 Rule 5

Never reuse an AES-GCM nonce with the same key.

🔴 Rule 6

k-anonymity is a privacy-preserving query technique, not encryption.

🔴 Rule 7

Don't send the complete password to an external breach-checking service.

🔴 Rule 8

Password generation must use strong randomness.

🔴 Rule 9

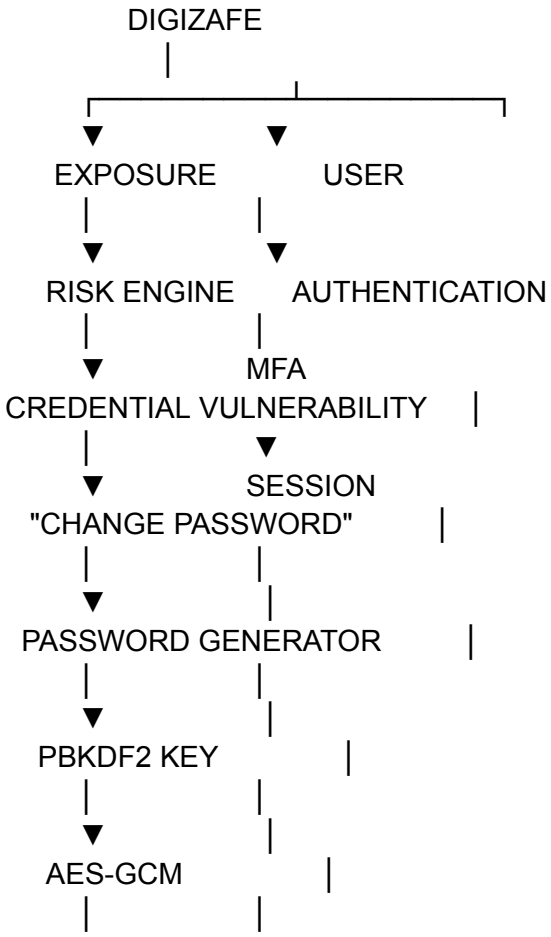
Risk scoring should be explainable and reproducible.

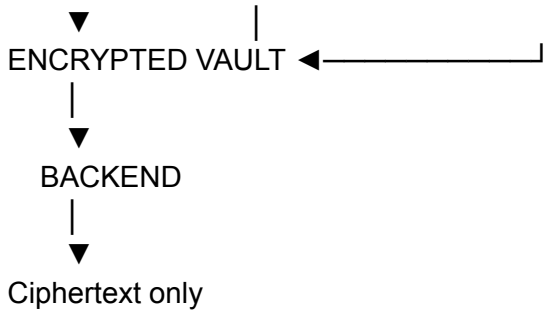
🔴 Rule 10

Detection is incomplete without actionable remediation.

FINAL MEMBER 4 MENTAL MODEL

Your entire contribution can be remembered as:





Your strongest one-line viva description:

> **"I worked on the Zero-Trust credential-protection layer that connects DigiZafe's risk assessment to secure password remediation through client-side PBKDF2/AES-GCM vault encryption and privacy-preserving password exposure verification."**